F

FeeFlow

FEES & PARENT PORTAL

Technical Design Document

Coaching-institute fee tracking system & parent portal

IsoCode Labs — Software Engineering Assessment · Prepared 9 November 2026

GitHub repository: **https://github.com/harika12choco/FeeFlow**

Live application: **Not deployed** — runs locally; not deployed or verified (see §1, §10)

All data shown is invented sample data for assessment purposes.

1. Project Information

REPOSITORY & LIVE APPLICATION

- **GitHub repository:** https://github.com/harika12choco/FeeFlow — the source is published here.
- **Live application URL: Not deployed.** The application has not been deployed or verified on any hosting platform; it runs locally only. This remains a placeholder until a real deployment is verified.

Project name

FeeFlow — a fee-tracking system for a coaching institute, with a dedicated parent portal.

Problem statement

Coaching institutes manage fees across many students, batches and centres. Accounts staff need an accurate ledger of what was billed, collected and is still outstanding, and need to chase dues without manual spreadsheets. Parents need a clear, trustworthy view of what they owe and a simple way to pay. The institute also wants early, non-punitive signals that a student may be at risk of dropping out, so a counsellor can intervene. The system must keep money data correct and auditable, restrict access by role, and never take automated financial action off the back of a predictive signal.

Solution overview

FeeFlow is a two-role web application. An **Accounts Executive** gets a dues dashboard, a per-student fee ledger, reminder management, a deterministic pro-rata refund calculator, and an AI-assisted dropout-risk review. A **Parent** gets a mobile-first portal showing their own child's outstanding amount, next due date, instalments, payment history, receipts, attendance and test scores, with a simulated "Pay Now" flow that generates a receipt and stops reminders for the paid instalment. The backend is the single source of truth for every balance and state transition; the frontend never marks a payment successful until the backend confirms it.

Item	Value
Product name	FeeFlow
Roles	Accounts Executive, Parent
GitHub repository URL	https://github.com/harika12choco/FeeFlow
Live application URL	Not deployed (local only, not verified) — placeholder
Backend (local)	`http://127.0.0.1:8077` · interactive docs at `/docs`
Frontend (local)	`http://localhost:5173`

2. Requirements and Scope

User roles

Role	Capabilities (enforced on the backend)
Accounts Executive	View all students and the dues dashboard; open any student's fee ledger; collect payments; generate/dispatch/cancel reminders; run the refund calculator and record/approve refund requests; run dropout-risk assessment and manage counsellor-review flags.
Parent	Access the portal for *their own linked student only*; view dues, instalments, payment history, receipts, attendance and test scores; initiate and confirm a simulated payment for their child's instalments.

Functional requirements

1. **Fee ledger** — per student: fee plan, discounts, instalments with due dates, payments, paid/outstanding balances, overdue amount and days overdue, and an overall status (paid / part-paid / overdue / upcoming).
2. **Parent portal** — the five-second summary (total outstanding + next due date) plus instalment breakdown, payment history, receipts, attendance and test scores.
3. **Payments** — per-instalment payment link; simulated flow of *initiate* → *confirm*; backend records the transaction, updates the balance, generates a receipt, and prevents duplicate processing.
4. **Reminders** — before-due / on-due / after-due reminders, in English & Hindi, created and scheduled deterministically, dispatched via a mocked provider, auto-cancelled when an instalment is fully paid, with full history.
5. **Refund calculator** — deterministic, configurable pro-rata calculation with a visible breakdown and clearly labelled assumptions; refund requests are recorded with a status workflow.
6. **Dropout-risk flag** — AI-assisted risk assessment that produces a counsellor-review task only, with a transparent rule-based fallback.
7. **Dues dashboard** — totals (billed, collected, outstanding, overdue), collection rate, dues by centre and by batch, ageing buckets, and high-risk students, with working filters.
8. **Security** — password hashing, JWT auth, role-based access, parent restricted to their own student, input validation, no secrets in the repo.

Non-functional requirements

- **Correctness & auditability of money** — all balances derive from one ledger module; refunds are deterministic and itemised.
- **Security by default** — authorization enforced server-side on every sensitive route; least-privilege by role.
- **Persistence** — durable storage via SQLAlchemy; SQLite for local/demo, PostgreSQL-compatible for production.
- **Clarity of UX** — every screen has loading, empty and error states; the parent's key number is readable in ~5 seconds.
- **Portability** — one-command backend bootstrap; standard Vite frontend.
- **Honesty of AI** — rule-based results are never presented as AI; the engine is transparent.

Assumptions made during development

- **Refund policy is not specified by the brief**, so it is treated as an explicit, configurable assumption (three policies; see §6-adjacent and §5). Each policy prints its own assumptions.

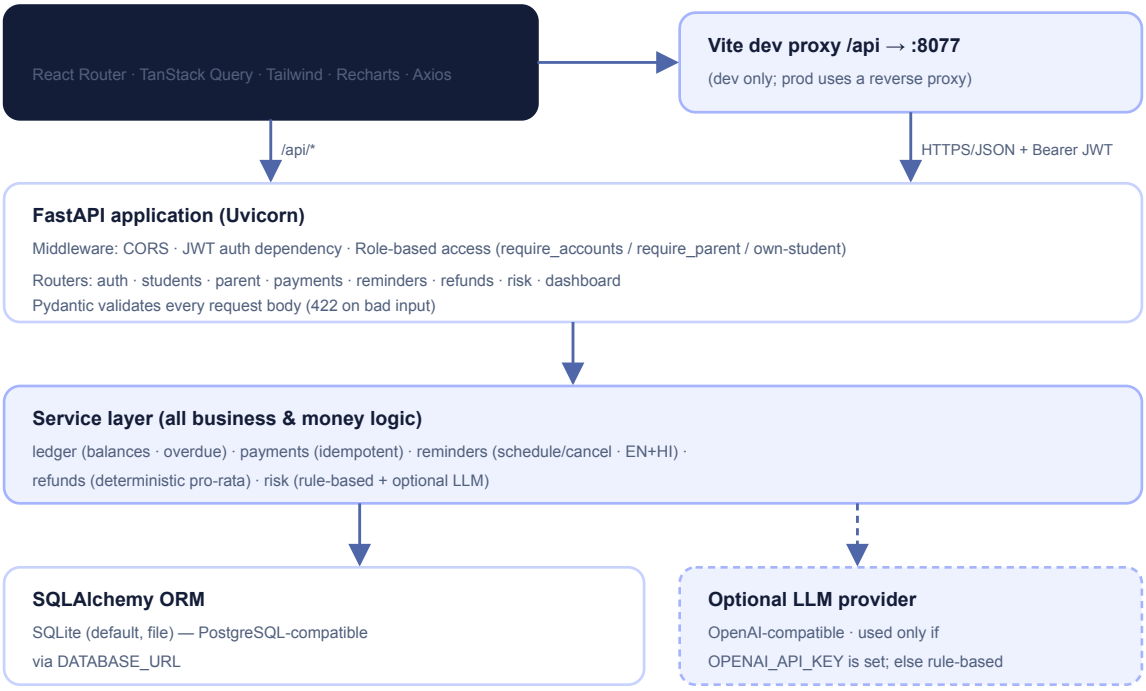
- **Payment gateway and messaging providers are out of scope**; they are simulated/mocked by design, while all persistence and state transitions are real.
- **Money is stored as a floating-point rupee amount** with explicit 2-dp rounding in the service layer (acceptable for whole-rupee demo data; a production system would use integer minor units or `Numeric`).
- **One parent account maps to one or more students**; the seeded demo links the demo parent to a single student (STU-0587).
- **A reminder "dispatch" marks records as sent** at or after their scheduled date — this simulates a scheduler/cron without a background worker.
- **All seeded records are fictional** and flagged `is_sample = true`; the UI shows a "Sample data" banner.

3. Architecture and Technical Decisions

Technology stack (as actually implemented)

Layer	Technologies	Notes
Frontend	React 18, TypeScript, Vite 6, Tailwind CSS 3, Recharts, lucide-react, React Router 6, TanStack Query 5, Axios	Custom shadcn-style UI primitives; charts via Recharts.
Backend	Python 3.12, FastAPI, SQLAlchemy 2.0, Pydantic v2, PyJWT, passlib (pbkdf2_sha256), Uvicorn	Service layer holds all money/risk logic.
Database	SQLite (default), PostgreSQL-compatible via `DATABASE_URL`	SQLAlchemy ORM; swap the URL for Postgres.
Auth	JWT bearer tokens; pbkdf2_sha256 password hashing; role-based dependencies	Parent restricted to own student.
AI (optional)	OpenAI-compatible chat endpoint via `OPENAI_API_KEY`	Falls back to a deterministic rule-based engine.
Tests	pytest + FastAPI TestClient	34 tests; unit + API-level.

System architecture



Solid arrows = request path. Dashed = optional/absent by default (rule-based fallback is the default risk path).

API communication

The SPA talks to the backend over JSON/HTTP. In development, Vite proxies `/api/*` to the backend at `127.0.0.1:8077`, so there are no CORS issues; the backend also enables CORS for the dev origin. Axios attaches

the JWT as a `Bearer` token on every request and clears it on a `401`. TanStack Query manages fetching, caching, and the loading/error states each screen renders.

Authentication & authorization

- **Login** (`POST /auth/login`) verifies the password hash and returns a signed JWT carrying the subject (email) and role.
- **Hashing** uses passlib `pbkdf2_sha256` (pure-Python, no native build), so passwords are never stored in plaintext.
- **Authorization** is enforced by FastAPI dependencies: `require_accounts`, `require_parent`, and `authorize_student_access()`, which allows accounts staff to any student but a parent only to their linked student.

Database & persistence

SQLAlchemy 2.0 models map to tables created on startup. The engine is chosen from `DATABASE_URL`: SQLite by default (a durable file, with `check_same_thread` disabled for the dev server), or any PostgreSQL URL in production. Money columns are `Float` with explicit rounding in the service layer; JSON columns store the refund breakdown and risk reasons.

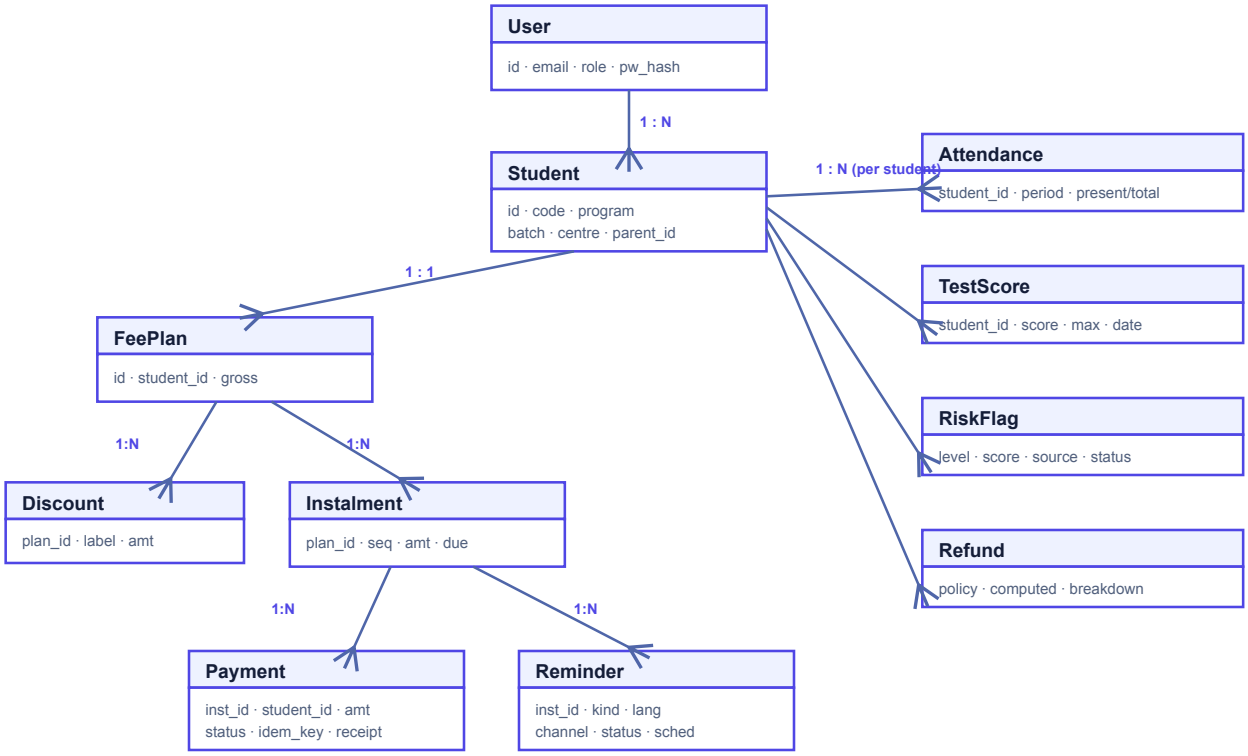
Why these technologies

- **FastAPI + Pydantic** — typed request validation, dependency-injected auth/RBAC, and auto-generated OpenAPI docs, which keep sensitive-operation guards explicit and testable.
- **SQLAlchemy with a SQLite→Postgres swap** — zero-setup local persistence that stays production-portable by changing one environment variable.
- **React + TypeScript + Vite** — fast DX and type safety across the API boundary; TanStack Query gives consistent loading/empty/error handling.
- **Tailwind + Recharts + lucide** — a small, consistent design system and dashboard charts without a heavy UI framework.
- **JWT + pbkdf2_sha256** — stateless auth and a dependency-free hash that avoids native-build fragility.
- **Rule-based-first risk engine** — deterministic and explainable by default; the LLM is strictly optional and transparently labelled.

4. Data Model

Eleven entities, defined in `backend/app/models.py`. Money is stored as `Float`; `reasons` and the refund `breakdown` are JSON columns.

Entity-relationship diagram



Crow's-foot points to the "many" side. Payment and Reminder also carry a denormalised `student_id` for fast per-student queries.

Entities & key fields

Entity	Key fields	Purpose / relationships
User	id, email (unique), hashed_password, full_name, role	Accounts or parent. A parent user links to one or more Students.
Student	student_code (unique), name, program, batch, centre, parent_user_id, course_start/end, is_sample	Centre of the model; owns one FeePlan and many behavioural records.
FeePlan	student_id (unique), name, gross_amount	One per student; gross before discounts.
Discount	fee_plan_id, label, amount	Absolute rupee reductions on a plan.
Instalment	fee_plan_id, seq, label, amount, due_date	Schedule rows; own Payments and Reminders.
Payment	instalment_id, student_id, amount, status, method, **idempotency_key (unique)**, receipt_no (unique), confirmed_at	Pending → confirmed; the unique idempotency key prevents duplicates.

Reminder	instalment_id, student_id, kind, channel, language, scheduled_for, status, message, sent_at, cancelled_at	before/on/after due; EN/HI; scheduled → sent | cancelled.
Refund	student_id, policy_key, reason, paid_amount, computed_refund, breakdown (JSON), status	Recorded request; requested → approved/rejected → processed.
RiskFlag	student_id, risk_level, score, reasons (JSON), recommended_action, confidence, source, model, status, counsellor_task	A counsellor-review task; source is `ai` or `rule_based`.
Attendance	student_id, period, present, total (→ % derived)	Monthly attendance; feeds risk.
TestScore	student_id, test_name, taken_on, score, max_score (→ % derived)	Assessment results; feed risk trend.

Key constraints & relationships

- **Idempotent payments** — `Payment.idempotency_key` is unique; `receipt_no` is unique and assigned only on confirmation.
- **One plan per student** — `FeePlan.student_id` is unique.
- **Cascades** — deleting a Student cascades to its FeePlan, Attendance, TestScores and RiskFlags; a FeePlan cascades to Discounts and Instalments; an Instalment cascades to its Payments and Reminders.
- **Parent linkage** — `Student.parent_user_id` is the single authority used to restrict parent access.
- **Outstanding is never stored** — balances, overdue amounts and days-overdue are always computed from confirmed payments against instalments, so the ledger cannot drift.

5. Core Application Workflows

5.1 Accounts executive login

1. Executive submits email + password → `POST /auth/login`.
2. Backend verifies the pbkdf2 hash and returns a JWT with role `accounts`.
3. Frontend stores the token, calls `GET /auth/me`, and routes to the Accounts Dashboard.

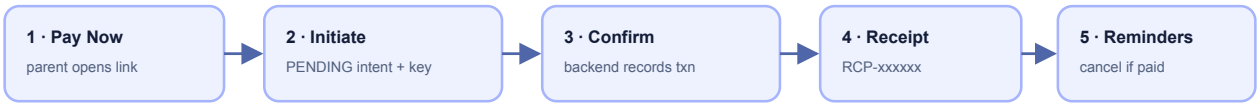
5.2 Parent login & portal access

1. Parent logs in; JWT carries role `parent`.
2. `GET /parent/portal` resolves the student via `parent_user_id` and returns that student's full ledger only.
3. Any attempt to read another student (`GET /students/{other}`) or pay another student's instalment returns **403**.

5.3 Fee ledger & outstanding balance

1. `GET /students/{id}` loads the plan, discounts and instalments.
2. The ledger service sums *confirmed* payments per instalment → paid, balance, status, days overdue.
3. Student-level net payable = gross − discounts; outstanding = net − total paid; overdue = sum of balances past due.

5.4 Payment initiation & confirmation (simulated)



Balance is NOT changed at step 2. Duplicate confirm → same receipt, no double-count.

1. `GET /payments/link/{instalment_id}` returns the payable balance.
2. `POST /payments/initiate` creates a **pending** payment with a client idempotency key; nothing is "paid" yet.
3. `POST /payments/confirm` marks it confirmed, stamps `confirmed_at`, and assigns a receipt number. Confirming the same payment twice is a no-op (no duplicate, same receipt).
4. Over-paying an instalment is rejected (**400**); the balance updates only from confirmed payments.
5. If the instalment becomes fully paid, its *scheduled* reminders are auto-cancelled.

5.5 Receipt generation & retrieval

On confirmation the backend assigns a sequential `RCP-NNNNNN` number. `GET /payments/{id}/receipt` returns the receipt (student, program, centre, instalment, amount, method, issued-at) for confirmed payments only; the frontend renders a printable receipt.

5.6 Reminders — generation, cancellation, payment-stop

1. `POST /reminders/generate` creates before-due (−3 days), on-due, and after-due (+3 days) reminders for unpaid instalments, in English or Hindi. Generation is idempotent (no duplicates).
2. `POST /reminders/dispatch` marks every scheduled reminder whose date has arrived as **sent** (mocked provider).
3. `POST /reminders/{id}/cancel` cancels a scheduled reminder manually.

4. **Payment-stop:** when an instalment is fully paid, its scheduled reminders move to **cancelled** (already-sent reminders are history and are not retro-cancelled).

5.7 Refund — calculation, request, approval

1. `GET /refunds/policies` lists the configurable policies, each with its assumptions.
2. `POST /refunds/calculate` returns an itemised, deterministic breakdown (never uses AI).
3. `POST /refunds` records a request (status `requested`) storing the full breakdown.
4. `POST /refunds/{id}/decision` moves it to `approved` / `rejected` / `processed`.

> **Refund policies (configurable assumptions).**
>
> - **Standard pro-rata (linear)** — refund proportional to unused course duration, minus a flat ₹2,500 admin fee.
> - **Pro-rata with non-refundable registration** — 10% registration retained, remainder pro-rata, minus ₹1,000.
> - **Slab-based** — 90/75/50/25/0% refund at 10/25/50/75/100% elapsed.

5.8 Dues dashboard

`GET /dashboard/stats` aggregates across students (with optional `centre` / `batch` / `program` filters): total billed, collected, outstanding, overdue; collection rate; dues by centre and by batch; ageing buckets (current / 1–15 / 16–30 / 31–60 / 60+); and open high/medium-risk students.

5.9 AI dropout-risk identification & counsellor task creation

1. `POST /risk/assess` (one student) or `POST /risk/assess-all` computes features: missed instalments, days overdue, attendance %, test-score trend.
2. The engine returns risk level, 0–100 score, reasons, recommended action and confidence, labelled `ai` or `rule_based`.
3. A **RiskFlag** is created — this is the counsellor-review task. It changes no fees, applies no penalty, and sends no message.
4. `POST /risk/flags/{id}/status` lets staff mark a flag `false_positive` / `resolved` / re-open.

6. AI Design

How a dropout-risk flag is generated (as implemented)

The service (`backend/app/services/risk.py`) extracts four signals per student — **missed instalments**, **days overdue**, **latest attendance %** and **test-score trend** (latest minus earliest). It then produces an assessment and the router persists a **RiskFlag** (the counsellor-review task).

Risk score, level, reasons, action, confidence

- **Score (0–100)** — additive points: overdue instalments (up to 30), days overdue (10/20/30 by severity), low attendance (12/25), declining/weak scores (7/15/+8), capped at 100.
- **Level** — high ≥ 60, medium ≥ 30, else low.
- **Reasons** — human-readable strings citing the actual numbers (e.g. "Attendance is low at 48.0%").
- **Recommended action** — level-specific guidance that explicitly says not to alter fees automatically.
- **Confidence & uncertainty** — confidence falls when input signals are missing, with a plain-language note (e.g. "1 of 2 behavioural signals were missing").

Rule-based fallback (the default)

The deterministic engine above is the **default** path. It is transparent and reproducible, and every result it produces is labelled `source = "rule_based"` with `model = null`. The UI shows a "Rule-based" badge accordingly. In the current build (no LLM key configured) every flag is rule-based — confirmed in testing (score 100/100, source rule_based, confidence 1.0 for the worst-case student).

Optional LLM integration (accurate description)

If `OPENAI_API_KEY` is set, the service may call an OpenAI-compatible chat endpoint with a strict JSON-schema prompt and the same four signals, at temperature 0. The system prompt forbids changing fees, penalties or messaging. The response is validated; on **any** error (missing key, network, bad JSON, invalid level) the code falls back to the rule-based engine. A successful LLM result is labelled `source = "ai"` with the model name. The build has no key configured, so this path is dormant by default.

Hard guardrails

> **A risk assessment may only create a counsellor-review task.** It must never change fees, trigger penalties, message parents, or make any automatic financial decision. This is enforced in the router (the only side effect is inserting a RiskFlag) and verified by a test that asserts the student's ledger is unchanged after an assessment.

Implemented vs. future (not overstated)

Implemented today	Future improvement
Deterministic rule-based scoring on four signals, with reasons/action/confidence.	Trained model on historical dropout outcomes; calibrated probabilities.
Optional LLM call with strict JSON output and transparent fallback.	Richer feature set (payment cadence, engagement, support tickets).
False-positive marking and counsellor-task lifecycle.	Feedback loop from counsellor outcomes back into scoring.

The system does not claim predictive accuracy; it surfaces explainable signals for a human counsellor to act on.

7. UI and Design Decisions

A premium, restrained education-finance aesthetic: white and light-grey surfaces, deep-navy typography, a single indigo accent, generous spacing, and carefully designed cards and tables. Charts use Recharts; icons use lucide. Every screen implements **loading, empty and error** states, and every visible button performs a real backend action.

Accounts-executive experience

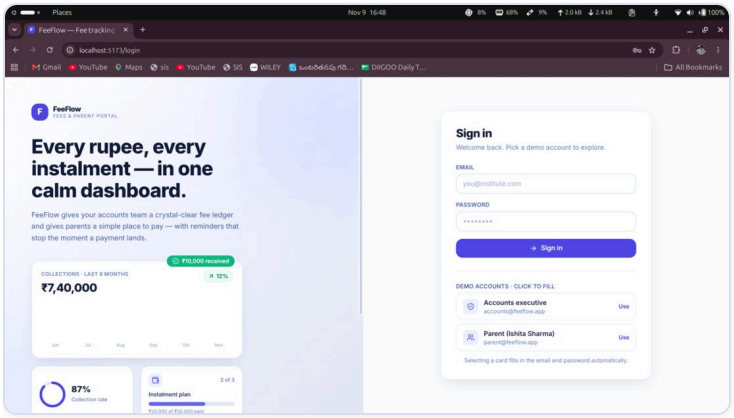
A left sidebar links the dashboard, student ledger, reminders, refund calculator and dropout-risk review. The dashboard leads with KPI cards and a collection-rate bar, then dues-by-centre/batch charts, ageing, and a high-risk list; filters (centre/batch/program) refine every figure. The ledger list is a sortable table with status and risk badges; the detail view shows instalments with per-row "collect payment", payment history, attendance and scores.

Parent experience (mobile-first)

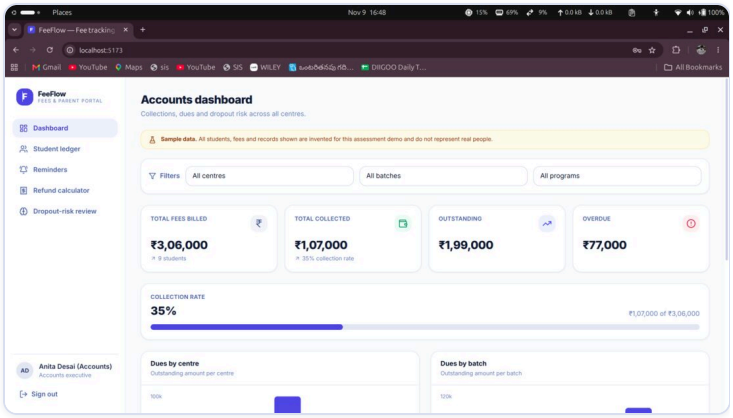
A dark hero card states the **total outstanding** and **next due date** first, with a prominent "Pay now" button — the key number is readable in ~5 seconds. Below it: instalments, payment history/receipts, attendance, test scores, and a bilingual reminder history. The payment dialog walks through amount → confirm → receipt, and never shows success before backend confirmation.

Screens from the running build (actual captures)

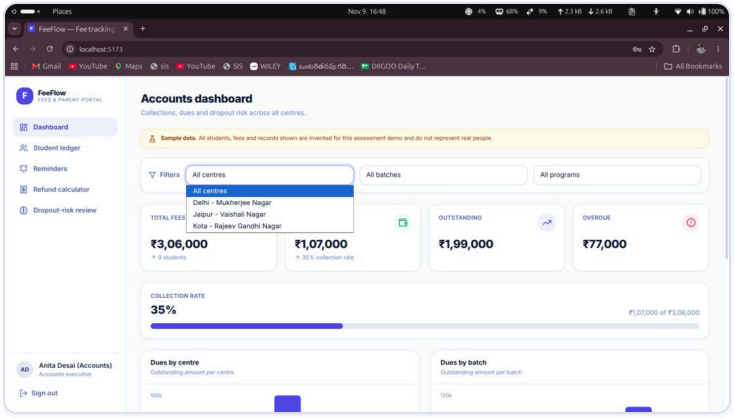
The following are real screenshots of the running application (sample data).



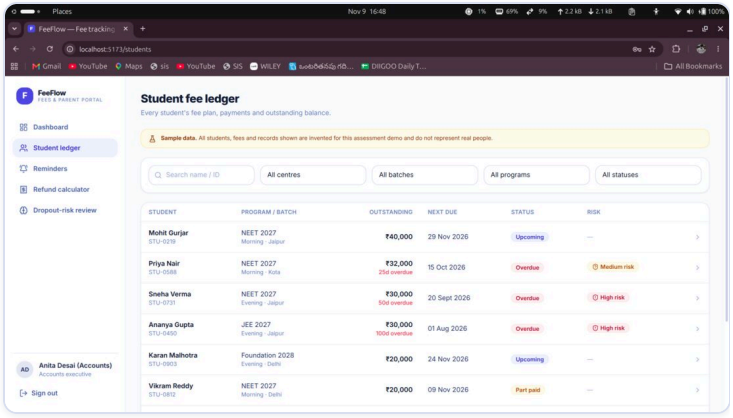
Login. Branding panel with a dashboard-style illustration; email/password form with one-click demo-account fill.



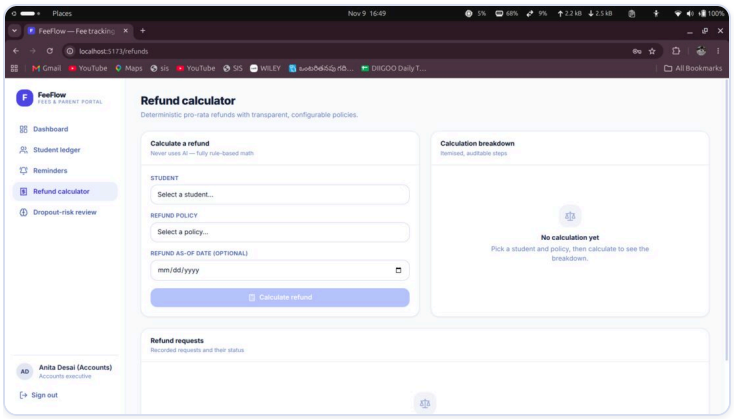
Accounts dashboard. KPI cards (billed / collected / outstanding / overdue), collection rate, and the "Sample data" banner.



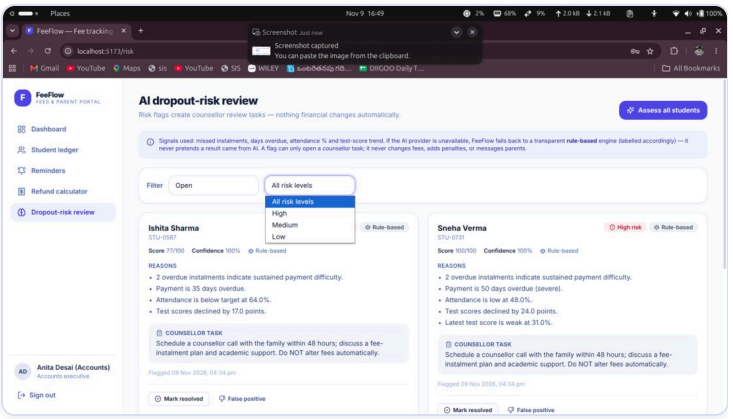
Dashboard filters. Centre filter open — figures recompute by centre / batch / program.



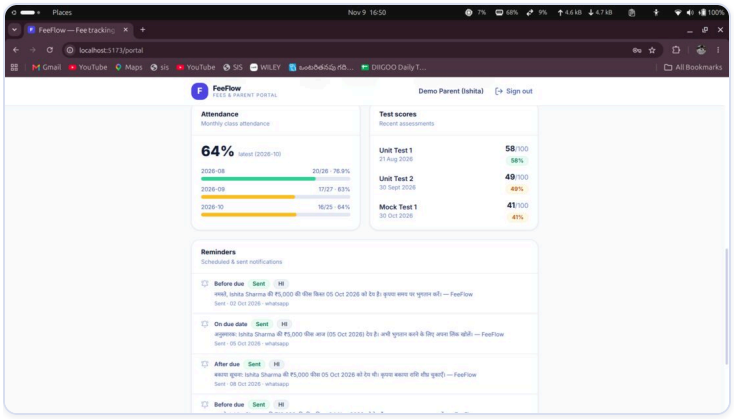
Student fee ledger. Per-student outstanding, next due, status and risk badges with search and filters.



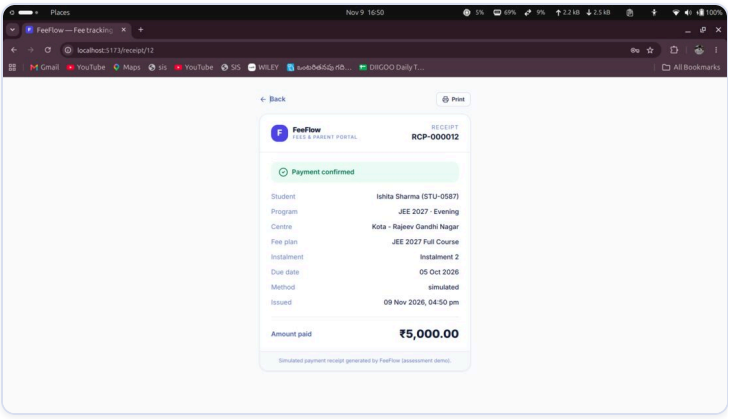
Refund calculator. Deterministic, "never uses AI"; empty state before a calculation.



AI dropout-risk review. Rule-based flags with score, confidence, reasons and the counsellor task; resolve / false-positive actions.



Parent portal. Attendance, test scores and a bilingual (Hindi) reminder history.



Payment receipt. Printable receipt (RCP-000012) generated after backend confirmation.

Additional screens exist (student-ledger detail, reminder management, the pay → confirm dialog, and the payment link); the set above covers each required screen.

8. Security Considerations

Control	Implementation (verified)
Role-based access control	`require_accounts` / `require_parent` dependencies. A parent receives **403** on all accounts routes; an accounts user receives 403 on the parent portal.
Parent restricted to own student	`authorize_student_access()` allows a parent only their linked student. Reading or paying another student returns **403** (own student → 200).
No cross-student access	Verified: parent → other student's detail and payment link both **403**; cross-student payment-initiate **403**.
Authentication & token handling	JWT bearer tokens signed with `JWT_SECRET`. Missing / tampered / malformed tokens all return **401**; Axios clears the token on 401.
Password hashing	passlib `pbkdf2_sha256`; plaintext passwords are never stored.
Input validation	Pydantic validates every body: invalid email → **422**, non-positive payment amount → **422**, over-payment → **400**.
Secrets management	Config via environment / `.env` (see `.env.example`). `.env` and `*.db` are git-ignored; no real `.env` is committed. The frontend bundle contains no API keys or secrets.
Demo-data handling	All records are fictional and flagged `is_sample = true`; the UI shows a "Sample data" banner. Demo accounts can reach only this fictional data.

Known limitations

- **JWT secret default** — the default `JWT_SECRET` is a development placeholder. A strong secret **must** be set via environment in any real deployment.
- JWTs are not revocable before expiry (no server-side session store); acceptable for this scope.
- No rate-limiting / brute-force lockout on login.
- The demo password for the fictional accounts is intentionally present in the client so the demo cards can auto-fill; this is acceptable only because it unlocks fictional data.

9. Testing and Validation

Backend tests use pytest with FastAPI's TestClient against an isolated throwaway SQLite database. **Executed result: 34 passed.** The frontend production build (`tsc` + `vite build`) also completed successfully.

Test file	Count	Covers
`test_instalments.py`	4	Instalment amounts/status/days-overdue; discounts; net/outstanding/overdue; upcoming vs overdue.
`test_payments.py`	6	Partial payments update balance; no success before confirm; duplicate-key returns same payment; confirm-twice no double-count; over-payment rejected.
`test_reminders.py`	6	Three reminders per unpaid instalment; none for paid; idempotent generation; **cancellation after full payment**; partial keeps reminders; dispatch of due.
`test_refunds.py`	6	Linear pro-rata; determinism; registration policy; slab boundary; floor at ₹0; policies expose assumptions.
`test_access.py`	7	Parent sees only own student; parent blocked from lists/dashboard; accounts can access all; unauthenticated 401; wrong password 401; cross-student payment 403.
`test_risk.py`	5	Rule-based fallback without API key; required fields; low-risk healthy student; confidence drops with missing signals; assessment creates a task with no financial side-effects; mark false-positive.

Tested workflows & security checks (verified live)

- Login (both roles), ledger list + detail, parent portal.
- Payment: initiate → pending (balance unchanged) → confirm → receipt; duplicate confirm produced no second receipt.
- Reminder stop-on-payment: a fully-paid instalment's three scheduled reminders moved to cancelled.
- Refund determinism: identical inputs produced identical output across three policies.
- Dashboard filters: a centre filter reduced the student count and recomputed dues.
- Risk: assessment created a counsellor task with the ledger unchanged; false-positive → resolved lifecycle.
- Security: RBAC both directions, parent-restriction, cross-student denial, and 401/422/400 validation.

Blocked / not performed (stated honestly)

- **Deployment not performed or verified** — no live-environment tests exist.
- **LLM (AI) path not exercised** — no API key is configured, so only the rule-based fallback was tested (by design).
- No automated frontend unit/E2E tests; the UI was validated manually and via the production build.
- Real payment-gateway and SMS/WhatsApp delivery are mocked and therefore not tested end-to-end with an external provider.

10. Scope Cuts and Limitations

- **Payments are simulated** — the initiate/confirm flow, idempotency, receipts and balance updates are real, but no real gateway is integrated.
- **Messaging is mocked** — SMS/WhatsApp/email providers are not called; reminder records, scheduling, dispatch state and cancellation are real.
- **Deployment is local/demo only** — not deployed or verified on any hosting platform; the live URL is a placeholder.
- **SQLite by default** — durable and PostgreSQL-compatible, but swap `DATABASE_URL` for Postgres in production.
- **AI is optional and rule-based by default** — no trained model; the LLM path is dormant unless a key is supplied.
- **Money as Float** — rounded deterministically for whole-rupee demo data; a production build should use integer minor units or `Numeric`.
- **Source published on GitHub** — github.com/harika12choco/FeeFlow. Not deployed to any runtime host.
- Default local port is **8077** (8000 was occupied in the build environment); the frontend dev proxy matches this.

11. Future Improvements

- **Production payment gateway** — integrate Razorpay/Stripe with webhooks; keep the existing idempotency and receipt model.
- **Real messaging** — WhatsApp Business / SMS / email via a provider, driven by a background scheduler (Celery/APScheduler) replacing the manual dispatch.
- **Hardened deployment** — containerise; PostgreSQL + managed secrets; HTTPS reverse proxy serving the built SPA and proxying `/api`; environment-specific `JWT_SECRET` and CORS.
- **Monitoring & audit** — structured logs, request tracing, metrics/alerts on collection rate and failed payments, and an immutable audit trail for financial actions and refund decisions.
- **Stronger auth** — refresh tokens + revocation, login rate-limiting/lockout, optional 2FA for accounts staff, and server-side demo-login that keeps all credentials off the client.
- **Improved risk analysis** — a model trained on historical outcomes with calibrated probabilities, a richer feature set, and a counsellor-feedback loop; keep the transparent rule-based engine as the explainable fallback.
- **Quality** — frontend unit/E2E tests (Vitest/Playwright), code-splitting to trim the bundle, and CI running tests + build on every push.

> **Submission status:**
>
> 1. **GitHub repository URL** — done: github.com/harika12choco/FeeFlow (shown on the cover and in §1).
> 2. **Live application URL** — still a placeholder ("Not deployed"); fill in only if a live instance is deployed and verified.